Problem solving, proved

Means-ends analysis, impasses, and the bugs of multi-column addition

Jeff Shrager

October 2026

A problem space

- **States** and **operators**; each operator has **preconditions** and an **effect** (Newell & Simon).
- An effect is a *distribution* over states, so an operator can include a probabilistic process, such as retrieving a fact from memory.
- One space, two control regimes:
  - **Means-ends analysis** (GPS): goals as differences, subgoals on unmet preconditions, backtracking.
  - **A decision cycle with impasses** (repair theory; Soar): rules propose an operator; a violated precondition is an impasse, repaired locally.

Means-ends analysis (GPS)

To achieve a goal, a list of conditions:

1. find the first condition that does not hold (the most important difference);
2. take the operators relevant to it from the **difference table**, in order;
3. for each: first achieve its preconditions (a **subgoal**), then apply it, then continue with the goal;
4. if it fails, **back up** and try the next.

Bounded by a depth. Proved once in the library, for any space.

Impasses and repairs (Brown & VanLehn 1980)

- A procedure is a set of rules; **deleting** rules models incomplete learning.
- An **impasse**: the proposed operator's precondition is violated.
- A **local** problem solver repairs it: no look-ahead, one action.
- **Repair heuristics** propose actions (Skip, Quit, Backup, Refocus, ...); **critics** veto some.
- Each surviving repair predicts a **bug**. Here, one is chosen at random.

Proved: an impasse is exactly a proposed operator whose precondition fails; without one, a step is just the operator's effect.

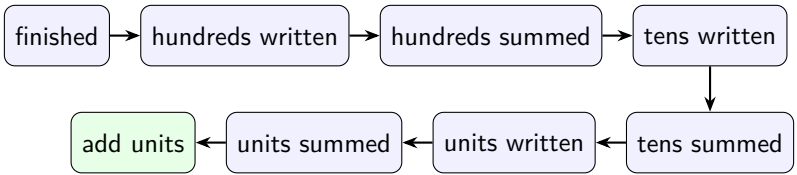
Multi-column addition as a problem space

State: the next column, the carry into it, its sum once known, what is written in each column, whether work has stopped.

Operator	Precondition	Effect
add column i	it is column i; sum unknown	sum := single-digit fact + carry
write	the sum is one digit	write it
write and carry	the sum is known	write its units; carry its tens
finish	past the hundreds	write any final carry
Repairs: skip (blank cell), quit, write the units digit only, write both digits		

The single-digit fact comes from *any* process: in the multi-digit model, a DOA trial (retrieval, fingers, counting).

GPS works units-first, unprompted



- The goal names no order: only *finished*. Finishing needs the hundreds written; writing needs the sum; adding a column needs the column to its right written, for its carry.
- Subgoaling works back to the units column, and the work runs units, tens, hundreds.
- **Proved for every problem**: GPS works units, tens, hundreds, and answers exactly.

The bridge: the solver is the column backup

- The multi-digit model's backup, `columns`, was a fixed procedure: units, tens, hundreds, with carrying written in.
- **Proved**: the problem solver with the full procedure answers *exactly* as `columns` does, for every problem and every single-digit process.
- So the solver **replaces** the backup, and every multi-digit theorem about it still holds: carrying adds no error; a column error is a single-digit error moved to its place; confident errors.

The proof regroups the solver's step-by-step chain of draws into the backup's three nested draws (associativity of probabilistic composition), then checks place value.

Delete the carry rule

- Without the carry rule, the procedure tries to *write* every column sum.
- A sum of 10 or more violates *write*'s precondition: an **impasse**.
- **Proved**: a problem whose columns all sum to less than 10 is still answered *exactly*, whatever the repairs. No impasse arises.

So a learner without carrying looks competent on every no-carry problem. Only carry problems reveal what is missing.

Four repairs, four bugs

Repair at the impasse	$15 + 17$ answered	probability
Skip (blank cell)	20	$1/4$
Quit	0	$1/4$
Write the units digit only	22 (no carry)	$1/4$
Write both digits	212	$1/4$
full procedure	32	1

- Repair theory's heuristics, carried over from subtraction, generate addition bugs, the no-carry bug among them.
- With no critics, the repair is chosen at random: a distribution over bugs.

Critics select the bug

- Critics against blank cells, quitting, and two digits in a cell leave one repair: write the units digit.
- **Proved, for every problem**: the answer is the columns added without carrying,

$$\sum_i \left((a_i + b_i) \bmod 10\right) \cdot 10^i.$$

A learner's bug is a joint product of what is missing (the carry rule) and what they will not do (the critics). A stable bug is predicted where the critics leave a single repair.

The same deficit, two control regimes

Hold the knowledge fixed: the same operators, the same carry operator deleted. Change only the control regime.

	no carry needed	a carry needed
Decision cycle with repair	exact	confident wrong (no-carry bug)
Means-ends analysis (GPS)	exact	**no answer**: it gives up

- Proved for every problem (`regimes_diverge`).
- A testable contrast: the same gap in knowledge shows as a systematic bug or as a stall, depending on how the learner controls their work.

What is proved, and what is not yet

For every problem / setting	On examples, or not yet
Means-ends analysis is sound (any space)	The four bugs on $15 + 17$ (one problem)
The bridge to `columns`	Soar's tie and conflict impasses
GPS units-first and exact	Chunking (next)
The no-carry bug under strict critics	Subtraction, repair theory's own domain
GPS gives up exactly on carry problems	
The two regimes diverge	

Next

- **Chunking** (Soar): the result of a subgoal becomes a new rule. In addition, a fact found by counting at a retrieval impasse becomes an association: DOA's own learning rule, seen from the problem-solving side.
- **Subtraction** and borrowing, where repair theory's bug data are.
- Laws comparing the regimes, matrix-style: *every subgoal arises from an impasse*, *learning only by chunking*, ...